

# Practical 3

```
In [2]: import numpy as np
from collections import defaultdict

GAMMA = 1.0
THETA = 1e-4

player_sums = range(12, 22)
dealer_cards = range(1, 11)
usable_aces = [True, False]
ACTIONS = [0, 1] # 0: stick, 1: hit

card_probs = {i: 1/13 for i in range(1, 11)}
card_probs[10] = 4/13

def get_dealer_probs(start_card):
    states = {(start_card if start_card != 1 else 11, start_card == 1): 1.0}
    final_outcomes = defaultdict(float)
    while states:
        new_states = defaultdict(float)
        for (curr_sum, curr_ace), state_prob in states.items():
            if curr_sum >= 17:
                final_outcomes[curr_sum] += state_prob
                continue
            for card, card_prob in card_probs.items():
                prob = state_prob * card_prob
                s, ace = curr_sum + (11 if card == 1 else card), curr_ace or (card
                if s > 21 and ace:
                    s -= 10
                    ace = False
                if s > 21:
                    final_outcomes[22] += prob
                else:
                    new_states[(s, ace)] += prob
            states = new_states
    return final_outcomes

DEALER_RESULTS = {card: get_dealer_probs(card) for card in dealer_cards}

def get_next_state(player_sum, usable_ace, card):
    new_sum = player_sum + (11 if card == 1 else card)
    new_ace = usable_ace or (card == 1)
    if new_sum > 21 and new_ace:
        new_sum -= 10
        new_ace = False
    return new_sum, new_ace

V = defaultdict(float)

# Value Iteration
while True:
    delta = 0
    for ps in player_sums:
        for dc in dealer_cards:
            for ua in usable_aces:
                state = (ps, dc, ua)
                v_old = V[state]

                # Stick
                expected_stick = sum(
                    prob * (1 if final_sum > 21 or ps > final_sum else -1 if ps < f
                    for final_sum, prob in DEALER_RESULTS[dc].items()
                )

                # Hit
                expected_hit = 0
                for card, prob in card_probs.items():
```

```

        ns, nua = get_next_state(ps, ua, card)
        if ns > 21:
            expected_hit += prob * -1
        elif ns < 12:
            expected_hit += prob * 0
        else:
            expected_hit += prob * (GAMMA * V[(ns, dc, nua)])

        V[state] = max(expected_stick, expected_hit)
        delta = max(delta, abs(v_old - V[state]))
    if delta < THETA:
        break

print("Value Iteration Completed")

# Build policy matrices
policy_usable = np.zeros((len(player_sums), len(dealer_cards)), dtype=int)
policy_noace = np.zeros((len(player_sums), len(dealer_cards)), dtype=int)

for i, ps in enumerate(player_sums):
    for j, dc in enumerate(dealer_cards):
        for ua in usable_aces:
            expected_stick = sum(
                prob * (1 if final_sum > 21 or ps > final_sum else -1 if ps < final
                for final_sum, prob in DEALER_RESULTS[dc].items()
            )
            expected_hit = sum(
                prob * (-1 if get_next_state(ps, ua, card)[0] > 21 else
                0 if get_next_state(ps, ua, card)[0] < 12 else
                GAMMA * V[get_next_state(ps, ua, card)])
                for card, prob in card_probs.items()
            )
            best_action = 0 if expected_stick >= expected_hit else 1
            if ua:
                policy_usable[i, j] = best_action
            else:
                policy_noace[i, j] = best_action

# Show terminal states and policies in matrix form
print("\n--- Terminal States ---")
print("Dealer bust (22) or dealer >= 17 → terminal outcome")

print("\nOptimal Policy (Usable Ace):")
print(policy_usable)

print("\nOptimal Policy (No Usable Ace):")
print(policy_noace)

print("\nLegend: 0 = Stick, 1 = Hit")

```

Value Iteration Completed

--- Terminal States ---

Dealer bust (22) or dealer  $\geq 17 \rightarrow$  terminal outcome

Optimal Policy (Usable Ace):

```
[[1 1 1 1 1 1 1 1 1 1]
 [1 1 1 1 1 1 1 1 1 1]
 [1 1 1 1 1 1 1 1 1 1]
 [1 1 1 1 1 1 1 1 1 1]
 [1 1 1 1 1 1 1 1 1 1]
 [1 1 1 1 1 1 1 1 1 1]
 [1 1 1 1 1 0 1 1 1 1]
 [1 0 0 0 0 0 0 0 1 1]
 [1 0 0 0 0 0 0 0 0 1]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]]
```

Optimal Policy (No Usable Ace):

```
[[1 0 0 0 0 0 1 1 1 1]
 [1 0 0 0 0 0 1 1 1 1]
 [1 0 0 0 0 0 1 1 1 1]
 [1 0 0 0 0 0 0 0 1 1]
 [1 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]]
```

Legend: 0 = Stick, 1 = Hit